

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

OxygenScript

PONDERACIÓN: 40 pts

 **Tiempo estimado: 35 hrs**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo.....	4
3. Enunciado del Proyecto.....	4
3.1. Arquitectura de la aplicación.....	4
3.1.1. Monolito - cliente /servidor.....	4
3.1.2. Diseño de la GUI.....	6
3.2. Generalidades del lenguaje.....	7
3.2.1. Identificadores.....	7
3.2.2. Comentarios.....	8
3.2.3. Tipos estáticos.....	9
3.3. Sintaxis del lenguaje.....	10
3.3.1. Bloques de sentencias.....	10
3.3.2. Variables.....	11
3.3.3. Inmutabilidad y Mutabilidad.....	12
3.3.4. Operadores de asignación.....	13
3.3.5. Operadores Aritméticos (suma, resta, multiplicación, división, módulo, negación unaria y tabla de tipos resultantes).....	14
3.3.6. Operadores relacionales (==, !=, >, >=, <, <=).....	16
3.3.7. Operadores lógicos (!, , &&).....	17
3.3.8. Sentencias de control de flujo (if, while, loop, match).....	17
3.3.9. Sentencias de transferencia (break, continue, return).....	19
3.3.10. Arreglos y Slices.....	21
3.3.11. Strings.....	23
3.3.12. Structs.....	25
3.3.13. Funciones y parámetros.....	26
3.3.14. Funciones embebidas (incluye de los arreglos).....	28
3.3.15. Función main.....	29
3.4. Reportes.....	30
3.4.1. Reporte de errores.....	30
3.4.2. Reporte tabla de símbolos.....	34
3.4.3. Reporte de AST.....	35
3.5. Alcance del proyecto.....	36
3.6. Entregables.....	36
4. Material de apoyo.....	37
5. Metodología.....	37
6. Recursos y herramientas a utilizar.....	38
7. Desarrollo de Habilidades Blandas.....	38
8. Cronograma.....	38
9. Requisitos para optar a la calificación.....	39

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Pensamiento crítico	Analizar diferentes alternativas de diseño y seleccionar las más adecuadas para la arquitectura del intérprete.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Aplica principios básicos de ingeniería, ciencias de computación y sistemas de información y comunicación, en la formulación y resolución adecuada de problemas complejos.
Competencia Específica	Demuestra pensamiento crítico, actitud investigativa y rigor analítico en el planteamiento y la resolución de problemas complejos.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Desarrollar un intérprete funcional para el lenguaje OxygenScript que implemente el análisis léxico, sintáctico y semántico, así como la ejecución de programas mediante un árbol de sintaxis abstracta (AST).
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Implementar el 100 % de las características definidas en la especificación del lenguaje para la fase del intérprete y verificar su funcionamiento mediante un conjunto de casos de prueba que validen la correcta ejecución de cada componente.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Desarrollar el proyecto utilizando herramientas de análisis léxico y sintáctico, estructuras de datos propias para el AST y las tecnologías definidas para el curso, siguiendo una arquitectura modular que facilite su implementación y mantenimiento.

Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Construir un intérprete que permita aplicar los conceptos fundamentales de compiladores y que sirva como base para futuras etapas del desarrollo de un compilador, fortaleciendo las competencias técnicas en el diseño e implementación de lenguajes de programación.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Completar el desarrollo del intérprete durante el período académico establecido, cumpliendo los hitos definidos para el análisis léxico, análisis sintáctico, análisis semántico, ejecución, pruebas y documentación, hasta la entrega final del proyecto.

2. Resumen Ejecutivo

El presente proyecto consiste en el desarrollo de un intérprete funcional para el lenguaje OxygenScript, un lenguaje académico con una sintaxis inspirada en Rust, diseñado para fines educativos en el estudio de compiladores. El objetivo es construir un sistema capaz de ejecutar la entrada en este lenguaje utilizando herramientas para el análisis léxico y sintáctico, permitiendo a los estudiantes comprender e implementar los principales componentes de un compilador.

3. Enunciado del Proyecto

3.1. Arquitectura de la aplicación

3.1.1. Monolito - cliente /servidor

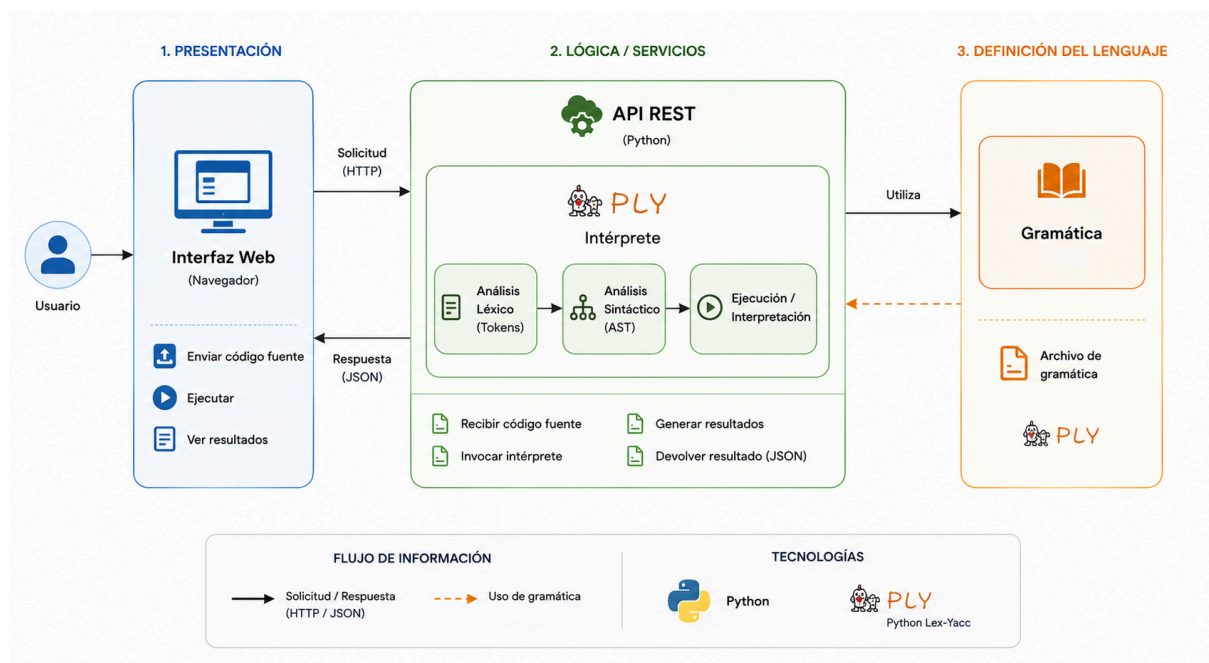


Figura 1. Ejemplo de Arquitectura

La aplicación se desarrollará bajo una arquitectura monolítica de tipo cliente/servidor, organizada en tres componentes principales que trabajan de forma coordinada para procesar y ejecutar programas escritos en OxygenScript.

El primer componente corresponde a la capa de presentación, implementada como una interfaz web accesible desde el navegador. Desde esta interfaz, el usuario puede escribir o cargar código fuente, ejecutar el programa y visualizar los resultados obtenidos, incluyendo la salida de consola y los reportes generados.

El segundo componente es la capa de lógica y servicios, expuesta mediante una API REST desarrollada en Python. Esta capa aloja el intérprete de OxygenScript, encargado de recibir el código fuente enviado desde el cliente, invocar el proceso de interpretación y devolver el resultado en formato JSON. Internamente, el intérprete se estructura en tres fases secuenciales: análisis léxico (generación de tokens), análisis sintáctico (construcción del árbol de sintaxis abstracta o AST) y ejecución/interpretación del programa.

El tercer componente corresponde a la definición del lenguaje, es decir, la gramática formal de OxygenScript especificada en un archivo de gramática y procesada mediante la herramienta PLY (Python Lex-Yacc). Este componente es utilizado directamente por el intérprete durante el análisis léxico y sintáctico, garantizando que el procesamiento del código fuente se ajuste a las reglas definidas para el lenguaje.

La comunicación entre el cliente y el servidor se realiza mediante solicitudes y respuestas HTTP en formato JSON, mientras que la relación entre el intérprete y la gramática se da a través del uso directo de las reglas léxicas y sintácticas definidas con PLY. Al tratarse de un monolito, todos estos componentes se despliegan como una sola aplicación, aunque internamente mantienen una separación clara de responsabilidades entre presentación, lógica de negocio y definición del lenguaje.

3.1.2. Diseño de la GUI

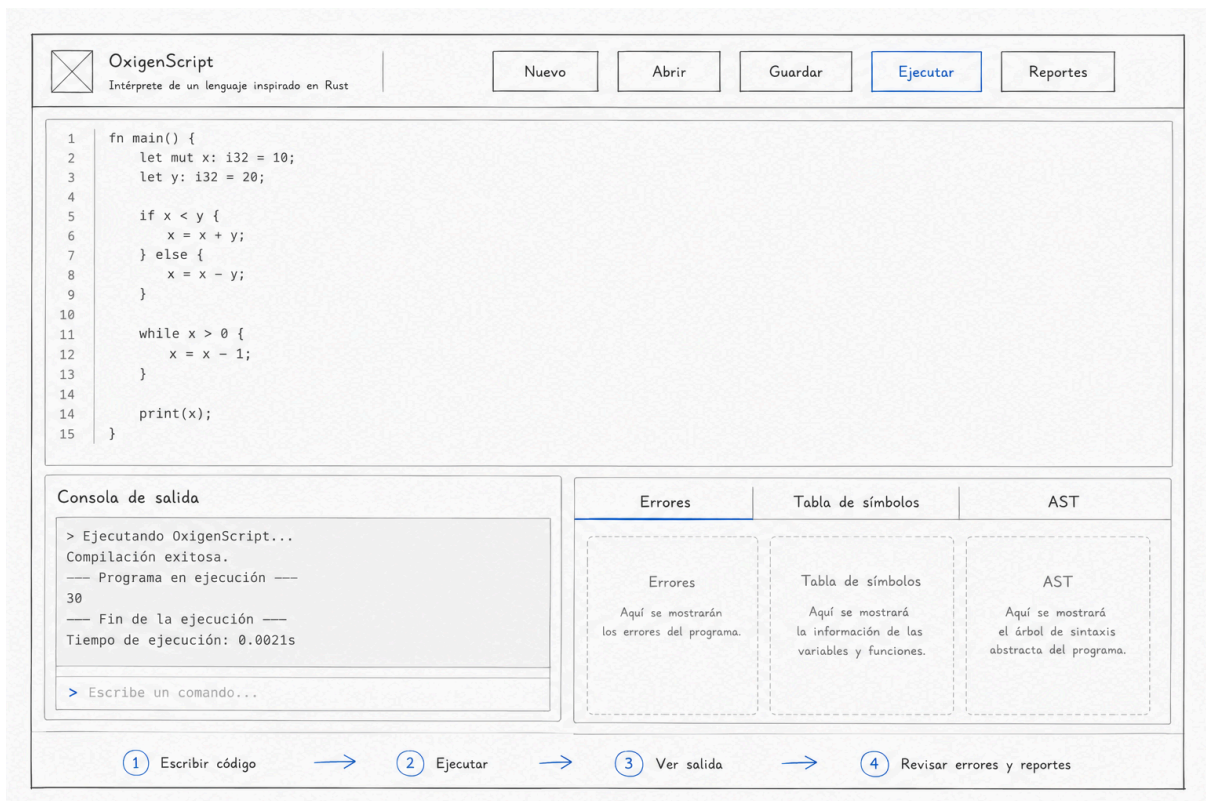


Figura 2. Ejemplo de interfaz gráfica.

La interfaz gráfica de OxigenScript está diseñada como un entorno de desarrollo simple e intuitivo, estructurado en cuatro áreas principales que guían al usuario a lo largo del flujo de trabajo: escribir código, ejecutarlo, ver la salida y revisar los reportes generados.

En la parte superior se ubica la barra de herramientas, que identifica la aplicación y agrupa las acciones principales sobre el archivo de código: crear un nuevo archivo, abrir uno existente, guardar cambios, ejecutar el programa y acceder a los reportes generados por el intérprete.

Debajo de la barra de herramientas se encuentra el editor de código, un área con numeración de líneas donde el usuario escribe o edita el programa fuente en OxigenScript. Este editor constituye el punto de entrada principal del flujo de trabajo.

En la parte inferior de la interfaz, la pantalla se divide en dos secciones complementarias:

- La consola de salida, ubicada del lado izquierdo, muestra el resultado de la ejecución del programa en tiempo real: mensajes de estado (por ejemplo, compilación exitosa), la salida generada por las instrucciones de impresión, el tiempo de ejecución y, si el usuario lo requiere, un espacio para escribir comandos adicionales.
- El panel de reportes, ubicado del lado derecho, organiza en pestañas la información generada durante el análisis del programa: Errores (detalle de los errores léxicos, sintácticos o semánticos detectados), Tabla de símbolos (información de las

variables y funciones registradas) y AST (representación gráfica del árbol de sintaxis abstracta del programa).

Este diseño permite que el usuario mantenga en una sola vista tanto el código fuente como los resultados de su ejecución y los reportes de análisis, favoreciendo un ciclo de prueba y depuración ágil.

3.2. Generalidades del lenguaje

3.2.1. Identificadores

```
<identifier> ::= ( <letter> | "_" ) ( <letter> | <digit> | "_" )*
```

Un identificador es el nombre que se le asigna a una variable, función, estructura o constante dentro de un programa. En OxygenScript, al igual que en Rust, un identificador debe cumplir con las siguientes reglas:

- Debe comenzar con una letra (a-z, A-Z) o un guión bajo (_).
- A partir del segundo carácter, puede contener letras, dígitos (0-9) y guiones bajos.
- No puede comenzar con un dígito.
- No puede coincidir con una palabra reservada del lenguaje (por ejemplo: fn, let, if, while, struct, return, entre otras).
- No se permiten espacios ni caracteres especiales como @, #, -, %, etc.

Case sensitive:

OxygenScript distingue entre mayúsculas y minúsculas (case sensitive), lo que significa que dos identificadores que difieren únicamente en el uso de mayúsculas o minúsculas son considerados diferentes.

Ejemplo:

```
fn main() {
    let edad = 20;
    let Edad = 25;
    let EDAD = 30;

    println!("{}", edad);
    println!("{}", Edad);
    println!("{}", EDAD);
}
```

Salida:

```
20
25
30
```


En el ejemplo anterior, edad, Edad y EDAD son tres identificadores distintos, ya que el lenguaje diferencia entre mayúsculas y minúsculas.

3.2.2. Comentarios

```
<comment> ::= "//" <any_char_except_newline>*
            | "/*" <any_char>* "*/"
```

Los comentarios son fragmentos de texto dentro del código fuente que el intérprete ignora durante el análisis léxico. Se utilizan para documentar el programa, explicar el propósito de una sección de código o desactivar temporalmente instrucciones, sin afectar la ejecución del mismo.

OxygenScript soporta dos tipos de comentarios:

- 1. Comentario de línea**

Comienza con // y se extiende hasta el final de la línea. Todo el texto después de // es ignorado por el intérprete.

- 2. Comentario de bloque**

Comienza con /* y termina con */. Puede abarcar una o varias líneas, y es útil para comentar bloques extensos de código o documentación más detallada.

Ejemplo:

```
fn main() {
    // Declaración de variables
    let x: i32 = 10;
    let y: i32 = 20; // Segundo operando

    /*
       Se calcula la suma de x e y
       y se almacena en la variable resultado
    */
    let resultado = x + y;

    println!("{}", resultado); // Imprime el resultado
}
```

Salida:

```
30
```

Consideraciones:

- Los comentarios no generan tokens durante el análisis léxico; son descartados antes de pasar a la fase de análisis sintáctico.
- Un comentario de bloque sin cierre (*/) debe ser reportado como un error léxico, ya que el analizador no podrá determinar dónde termina el comentario.

3.2.3. Tipos estáticos

```
<type> ::= "i32"
        | "f64"
        | "bool"
        | "char"
        | "String"
        | "[" <type> ";" <num> "]"
```

OxygenScript es un lenguaje de tipado estático, lo que significa que el tipo de cada variable debe conocerse en tiempo de compilación, ya sea porque el programador lo declara explícitamente o porque el intérprete lo infiere a partir del valor asignado. Una vez que una variable ha sido declarada con un tipo determinado, no es posible asignarle posteriormente un valor de un tipo diferente sin que esto genere un error semántico.

Declaración explícita e inferencia de tipos:

```
let edad: i32 = 20;    // Tipo explícito
let nombre = String::from("Ana"); // Tipo inferido: String
```

En el primer caso, el tipo `i32` se indica de forma explícita mediante la anotación `: i32`. En el segundo caso, el intérprete infiere el tipo `String` a partir del valor asignado (`String::from(...)`).

Tabla de tipos de datos:

Tipo	Categoría	Descripción	Valor por defecto
<code>i32</code>	Entero con signo	Representa cualquier número entero (positivo, negativo o cero). Tipo entero por defecto del lenguaje.	0
<code>f64</code>	Punto flotante	Representa números decimales de doble precisión. Tipo decimal por defecto del lenguaje.	0.0
<code>bool</code>	Booleano	Representa verdadero o falso.	false
<code>char</code>	Carácter	Representa un único carácter Unicode.	—
<code>String</code>	Cadena de texto	Cadena de texto dinámica.	""
<code>[T; N]</code>	Arreglo	Colección de tamaño fijo de N elementos del mismo tipo T.	—

Ejemplo:

```
fn main() {
```

```

// Variables con valor asignado explícitamente
let entero: i32 = 10;
let decimal: f64 = 3.14;

// Variables declaradas sin valor inicial (usan el valor por defecto)
let contador: i32;
let promedio: f64;
let activo: bool;
let mensaje: String;

println!("{}", entero);
println!("{}", decimal);

println!("{}", contador); // 0
println!("{}", promedio); // 0.0
println!("{}", activo); // false
println!("{}", mensaje); // "" (cadena vacía)
}

```

Salida:

```

10
3.14
0
0.0
false

```

3.3. Sintaxis del lenguaje

3.3.1. Bloques de sentencias

Un bloque de sentencias es una secuencia de declaraciones e instrucciones encerrada entre llaves ({}), que define un ámbito (scope) propio para las variables declaradas dentro del mismo. Los bloques se utilizan para agrupar instrucciones, controlar el flujo del programa y delimitar la visibilidad de identificadores.

Un bloque puede aparecer en funciones, estructuras de control (if, while, loop, for, match). Las variables declaradas dentro de un bloque solo son accesibles dentro de ese bloque y de sus sub-bloques.

```

<bloque> ::= "{" <instruccion>* "}"

```

Ejemplo:

```
fn main() {
    // Bloque de la función main
    let entero: i32 = 10;

    if entero > 5 {
        let entero_bloque: i32 = entero * 2;
        println!("{}", contador); // 0
    }
}
```

3.3.2. Variables

El lenguaje permite la declaración y uso de variables para almacenar valores de distintos tipos primitivos. Cada variable debe declararse antes de ser utilizada y está asociada a un tipo estático, el cual se determina en tiempo de compilación. Las variables poseen un alcance léxico, limitado al bloque en el que fueron declaradas.

Las variables pueden declararse mediante una sintaxis similar a Rust, por lo que pueden ser mutables o inmutables y pueden inicializarse opcionalmente al momento de declararlas, variables no inicializadas no pueden ser utilizadas.

Las variables declaradas pueden ser sombreadas, independientemente de su inmutabilidad, al declarar nuevamente con un id idéntico al anterior.

```
<var_decl> ::= <let_decl> <id> : <type> = <exp>
            | <let_decl> <id> = <exp>
            | <let_decl> <id> : <type>
            | <let_decl> <id>

<let_decl> ::= let
            | let mut
```

Ejemplo:

```
fn main() {
    // Variables con valor asignado explícitamente y tipo definido
    let entero: i32 = 10; // Esta variable no puede modificarse
    let mut impar: i32 = 3; // Esta variable puede modificarse

    // Variables declaradas sin valor inicial (usan el valor por defecto)
```

```

let contador: i32;
let mut termometro: i32;

println!("{}", entero);
println!("{}", impar);
println!("{}", termometro);    // 0
println!("{}", contador);      // 0.0

// Ejemplo de Sombreo (shadowing)
let var_sombreada: i32 = 10; // Esta variable no puede modificarse
println!("{}", entero);    // 10
// al redeclarar la variable, sobreescribiremos su valor

let var_sombreada: f64 = 3.14;
println!("{}", entero);    // 3.14
}

```

3.3.3. Inmutabilidad y Mutabilidad

En este lenguaje, al igual que en Rust, las variables son inmutables por defecto. Esto significa que, una vez asignado un valor a una variable, dicho valor no puede modificarse durante la ejecución del programa. Este comportamiento ayuda a escribir programas más seguros y reduce errores relacionados con cambios accidentales en los datos.

Cuando se desea que una variable pueda cambiar su valor, es necesario indicarlo explícitamente utilizando la palabra reservada `mut` al momento de su declaración.

Variables inmutables

Una variable declarada sin la palabra reservada `mut` solo puede recibir un valor durante su inicialización. Cualquier intento de modificarla posteriormente deberá producir un error semántico.

Ejemplo:

```

fn main() {
    let edad = 20;

    println!(edad);

    edad = 21;
}

```

Salida:

20

[Error Semántico] Línea 6, Columna 5
No es posible modificar una variable inmutable.

Variables mutables

Si una variable debe cambiar su valor durante la ejecución, deberá declararse utilizando la palabra reservada `mut`. Una variable mutable puede ser reasignada tantas veces como sea necesario, siempre que el nuevo valor sea compatible con su tipo de dato.

Ejemplo:

```
fn main() {
    let mut contador = 0;

    println!(contador);

    contador = 10;

    println!(contador);
}
```

Salida:

```
0
10
```

En este caso, la variable `contador` fue declarada como mutable mediante la palabra reservada `mut`, por lo que su valor puede modificarse después de su inicialización sin producir ningún error.

3.3.4. Operadores de asignación

Los operadores de asignación permiten asignar o actualizar el valor de una variable a partir de una expresión. El lenguaje soporta el operador de asignación simple y un conjunto limitado de operadores de asignación compuesta, los cuales combinan una operación aritmética o lógica con una asignación. El operando izquierdo debe ser una ubicación válida de almacenamiento, y el operando derecho debe ser una expresión compatible de tipo.

Operador	Descripción	Ejemplo
<code>=</code>	Asignación simple	<code>x = 5</code>

<code>+=</code>	Suma y Asignación	<code>x += 5</code>
<code>-=</code>	Resta y Asignación	<code>x -= 5</code>
<code>*=</code>	Multiplicación y Asignación	<code>x *= 5</code>
<code>/=</code>	División y Asignación	<code>x /= 4</code>
<code>%=</code>	Módulo y Asignación	<code>z %= 5</code>

3.3.5. Operadores Aritméticos (suma, resta, multiplicación, división, módulo, negación unaria y tabla de tipos resultantes)

Los operadores aritméticos permite realizar operaciones matemáticas básicas sobre valores numéricos y, en ciertos casos, operaciones definidas sobre cadenas. El lenguaje soporta suma, resta, multiplicación, división y módulo.

Los operandos deben ser expresiones cuyos tipos sean compatibles con la operación, de otro caso el resultado será **None**, debiendo registrarse el evento en la tabla de Errores como un error semántico. El tipo de resultado se determina en tiempo de ejecución mediante reglas de promoción de tipos.

Ejemplo:

```
fn main() {
    let a: i32 = 10;
    let b: f64 = 2.5;
    let nombre = String::from("Ana");

    let resultado1 = a + b;           // Correcto (promoción de tipos)
    let resultado2 = a * 5;          // Correcto
    let resultado3 = a + nombre;     // Error semántico

    println!(resultado1);
    println!(resultado2);
    println!(resultado3);
}
```

Salida:

```
[Error Semántico] Línea 7, Columna 24
No es posible aplicar el operador '+' entre los tipos i32 y String.

12.5
50
```

None

Suma:

+	i32	f64	bool	char	string			
i32	i32	f64						
f64	f64	f64						
bool								
char								
string					string			

Resta:

-	i32	f64	bool	char	string
i32	i32	f64			
f64	f64	f64			
bool					
char					
string					

Multiplicación:

*	i32	f64	bool	char	string
i32	i32	f64			
f64	f64	f64			
bool					
char					
string		string*			

*nota: 2:int32 * "string" = "stringstring", multiplicar int por string daría como resultado el string repetido n veces.

División:

/	i32	f64	bool	char	string
i32	i32	f64			
f64	f64	f64			
bool					
char					
string					

Módulo:

%	i32	f64	bool	char	string			
i32	i32	f64						
f64	f64	f64						
bool								
char								
string								

3.3.6. Operadores relacionales (==, !=, >, >=, <, <=)

Los operadores relacionales permiten comparar dos valores y producen como resultado un valor booleano (bool). Ambos operando deben ser de tipos compatibles según las reglas del lenguaje. La validez de una comparación y su tipo resultante se determinan en tiempo de ejecución.

Igualdad y desigualdad:

== / !=	i32	f64	bool	char	string
i32	bool	bool		bool	
f64	bool	bool			
bool			bool		
char	bool			bool	
string					bool

Mayor, menor, mayor o igual, menor o igual:

<, >, <=, >=	i32	f64	bool	char	string
i32	bool	bool		bool	
f64	bool	bool			
bool			bool		
char	bool			bool	
string					bool

3.3.7. Operadores lógicos (!, ||, &&)

Los operadores lógicos permiten combinar o negar expresiones booleanas y producen como resultado un valor de tipo bool. Estos operadores sólo están definidos para operandos de tipo bool. La validez de las operaciones se verifica en tiempo de ejecución.

Los operadores lógicos a utilizar son:

- negación (NOT, !)
- conjunción (AND, &&)
- disyunción (OR, ||)

Operador de Corto Circuito:

Los operadores lógicos binarios && y || deben evaluarse utilizando evaluación de corto circuito:

- En una expresión a && b, si a evalúa a false, la expresión b no se evalúa.
- En una expresión a || b, si a evalúa a true, la expresión b no se evalúa.

Esta restricción es obligatoria y forma parte de la semántica del lenguaje, ya que permite evitar evaluaciones innecesarias y efectos no deseados.

```
fn saludar(valor: i32) -> bool {
    println!("Hola, evaluando");
    return x > 0;
}

fn main() {
    // Variables con valor asignado explícitamente y tipo definido
    let flag: bool = false;

    flag && saludar(10); // La funcion no se ejecuta, dado que flag es falso
}
```

3.3.8. Sentencias de control de flujo (if, while, loop, match)

if:

La sentencia if permite la ejecución condicional de un bloque de sentencias en función del resultado de una expresión booleana.

```
<if_stmt> ::= <if> <cond> { <stmts> }
           | <if> <cond> { <stmts> } <else> { <stmts> }
```

- La condición debe ser tipo bool.
- No se permiten paréntesis alrededor de la condición.
- else if se modela como una continuación del else

```
fn main() {
    // Variables con valor asignado explícitamente y tipo definido
    let flag: bool = false;

    if flag {
        /// esto es verdadero
    } else if flag == false {
        /// esto es un if else
    } else {
        // este es un else
    }
}
```

match:

La sentencia match selecciona un bloque de ejecución entre múltiples alternativas basadas en el valor de una expresión.

Los valores a evaluar serán específicos y se deberá incluir una opción comodín que maneje todos los casos que no coincidan.

```
<match_stmt> ::= <match> <expr> { <stmts>
    <expr> => <stmts> ,
    _ => <stmts>,
}
```

```
fn main() {
    // Ejemplo de un match
    let number: i32 = 32;

    match number {
        1 => println!("UNO!"), // cuando number es igual a 1
        2  => println!("DOS!"), // cuando number es igual a 2
        _  => println!("nada más!"), // Comodín para todo valor no
    }
}
```

```

    }
}

```

loop:

Es una estructura que indica la realización de un ciclo infinito, que puede ser manipulado a través de las instrucciones break y continue.

```

<loop_stmt> ::= <loop> { <stmts> }
              | <tag> : <loop> { <stmts> }

```

```

fn main() {
    // un loop natural
    loop {
        // Instrucciones
    }

    // un loop anidado con etiquetas
    'outer: loop {
        // Instrucciones
        'inner: loop {
            // Instrucciones
            // se pueden utilizar instrucciones como break 'inner;
        }
    }
}

```

while:

Es una estructura de iteración que ejecuta un grupo de instrucciones mientras la condicion sea verdadera.

```

<loop_stmt> ::= <while> <cond> { <stmts> }

```

```

fn main() {

    while n == true {
        // Instrucciones
    }

}

```

3.3.9. Sentencias de transferencia (break, continue, return)

break:

Finaliza inmediatamente la ejecución de un bucle. Opcionalmente puede recibir una etiqueta como parametro y finalizar el bucle asociado a esa etiqueta.

```
fn main() {
    // un loop natural
    loop {
        // sale del bucle
        break;
    }

    // un loop anidado con etiquetas
    'outer: loop {
        // Instrucciones
        'inner: loop {
            // Finaliza el bucle 'outer, finalizando la ejecución de
            todo lo que este adentro
            break 'outer;
            println!("esto no se ejecuta");
        }
    }
}
```

continue:

Interrumpe la iteración actual de un bucle y continua con la siguiente.

```
fn main() {
    // un loop natural
    loop {
        // sale del bucle
        continue;
    }

    // un loop anidado con etiquetas
    'outer: loop {
        // Instrucciones
    }
```

```

        'inner: loop {
            // Finaliza el bucle 'outer, finalizando la ejecución de
            todo lo que este adentro
            continue 'outer;
            println!("esto no se ejecuta");
        }
    }
}

```

return:

Finaliza la ejecución de un bloque de instrucciones y devuelve opcionalmente un valor.

```

fn unNumero() -> i32 {
    println!("Hola, evaluando");
    // Se retorna un valor al finalizar la funcion.
    return 16;
}

fn main() {
    //se asigna el valor regresado por la funcion.
    let numero: i32 = unNumero();

    println!("El numero es {}", numero); // numero = 16
}

```

3.3.10. Arreglos y Slices

Un arreglo es una colección de objetos del mismo tipo **t**, almacenado en memoria contigua. Los arreglos se crean usando corchetes cuadrados **[]**, y su longitud, que se conoce en el momento de la compilación, es parte de su anotación de tipo **[T; longitud]**.

```

<array_decl> ::= <let_decl> <id> = [ <exp_list> ]
               | <let_decl> <id> : [ <type> ; <num> ] = [ <exp_list> ]
               | <let_decl> <id> = [ <exp> ; <num> ]

```

Ejemplo:

```

fn main() {
    // Declaración explícita (estática)
    let numeros: [i32; 5] = [10, 20, 30, 40, 50];
}

```

```

// Declaración por inferencia de tipos
let edades = [18, 20, 22];

// Acceso por índice
println!("Primer elemento: {}", numeros[0]);
println!("Tercer elemento: {}", numeros[2]);

// Cantidad de elementos
println!("Longitud de numeros: {}", numeros.len());
println!("Longitud de edades: {}", edades.len());

// Recorrer el arreglo con while
println!("\nElementos de numeros:");
let mut i = 0;

while i < numeros.len() {
    println!("{}", numeros[i]);
    i = i + 1;
}

// Slice (porción del arreglo)
let parte = &numeros[1..4];
println!("\nSlice: {:?}", parte);

// Acceso a otro arreglo
println!("Primera edad: {}", edades[0]);
}

```

Salida:

```

Primer elemento: 10
Tercer elemento: 30
Longitud de numeros: 5
Longitud de edades: 3

Elementos de numeros:
10
20
30
40
50

Slice: [20, 30, 40]
Primera edad: 18

```

Un slice es una vista o porción de un arreglo. A diferencia de un arreglo, un slice no almacena los datos, sino que hace referencia a una parte de un arreglo existente. Se crea utilizando el operador de rango (..) y el símbolo &, que indica un préstamo (borrow) del arreglo original. Esto permite trabajar con una sección del arreglo sin copiar sus elementos.

```
let numeros = [10, 20, 30, 40, 50];
let parte = &numeros[1..4];
```

El slice **parte** contiene los elementos: **[20, 30, 40]**

En este caso:

- 1 es el índice donde inicia el slice (incluido).
- 4 es el índice donde termina (no incluido).
- El arreglo original numeros permanece sin cambios.

3.3.11. Strings

```
<stringDecl> ::= <let_decl> <id> = String :: from ( <string_literal> )
                | <let_decl> <id> = String :: new ( )
```

Un String es una cadena de texto dinámica. String puede crecer o modificarse durante la ejecución del programa. Para ello, la variable debe declararse como mutable (mut). Entre sus operaciones más comunes se encuentran agregar texto, obtener su longitud y recorrer su contenido.

Ejemplo:

```
fn main() {
    // Crear un String
    let saludo = String::from("Hola Mundo desde Rust");

    // Mostrar el contenido
    println!("Cadena: {}", saludo);

    // Longitud de la cadena
    println!("Longitud: {}", saludo.len());

    // Reemplazar una parte del texto
    let nuevo_saludo = saludo.replace("Rust", "Compiladores");
    println!("Nueva cadena: {}", nuevo_saludo);

    // Recorrer las palabras con while
    println!("\nPalabras:");
    let palabras: Vec<&str> = saludo.split_whitespace().collect();
```



```

let mut i = 0;
while i < palabras.len() {
    println!("{}", palabras[i]);
    i = i + 1;
}
}

```

Salida:

```

Cadena: Hola Mundo desde Rust
Longitud: 21
Nueva cadena: Hola Mundo desde Compiladores

Palabras:
Hola
Mundo
desde
Rust

```

Literales y secuencias de escape:

Además de crear cadenas de texto con **String::from()**, Rust permite utilizar secuencias de escape para representar caracteres especiales dentro de un texto. Estas secuencias comienzan con una barra invertida (****) y permiten incluir saltos de línea, comillas dobles o barras invertidas sin alterar la sintaxis del programa.

En el ejemplo, **\n** inserta un salto de línea, **\"** permite escribir comillas dobles dentro de la cadena y **** representa una barra invertida (****).

Cuando una cadena contiene muchas barras invertidas o comillas, escribir todas las secuencias de escape puede resultar poco práctico. Para estos casos, Rust ofrece las raw strings, las cuales se escriben anteponiendo **r** a la cadena. En una raw string los caracteres se interpretan literalmente, por lo que no es necesario escapar las barras invertidas. Si la cadena contiene comillas dobles, se pueden utilizar delimitadores con **#**, por ejemplo **r#"..."#**, para que las comillas formen parte del texto sin necesidad de escaparlas.

Ejemplo:

```

fn main() {
    // Secuencias de escape
    let mensaje = String::from("Hola\nRust");
    let comillas = String::from("Él dijo: \"Hola\"");
    let ruta = String::from("C:\\Users\\Diego");

    println!("{}", mensaje);
    println!("{}", comillas);
    println!("{}", ruta);
}

```

```
// Raw Strings
let ruta_raw = String::from(r"C:\Users\Diego");
let texto = String::from(r#"Él dijo: "Hola Rust"");

println!("{}", ruta_raw);
println!("{}", texto);
}
```

Salida:

```
Hola
Rust
Él dijo: "Hola"
C:\Users\Diego
C:\Users\Diego
Él dijo: "Hola Rust"
```

3.3.12. Structs

```
<struct_decl> ::= struct <id> { <field_list> }

<field> ::= <id> : <type>

<struct_init> ::= <id> { <value_list> }
```

Una estructura (struct) es un tipo de dato definido por el usuario que permite agrupar varios valores relacionados bajo un mismo nombre. Cada uno de estos valores se denomina campo y puede ser de un tipo diferente.

Por ejemplo, una persona puede representarse mediante un nombre y una edad:

```
struct Person {
    name: String,
    age: u8,
}
```

Para crear una instancia de una estructura se asigna un valor a cada uno de sus campos.

```
let persona = Person {
    name: String::from("Pedro"),
    age: 27,
};
```

Los campos de una estructura se acceden mediante el operador punto (.).

```
println!("{}", persona.name);  
println!("{}", persona.age);
```

Una estructura también puede utilizar otras estructuras como tipo de sus campos. Esto permite representar datos más complejos de forma organizada.

```
struct Point {  
    x: f32,  
    y: f32,  
}  
  
struct Rectangle {  
    top_left: Point,  
    bottom_right: Point,  
}
```

Ejemplo:

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
struct Rectangle {  
    position: Point,  
}  
  
fn main() {  
    let rectangulo = Rectangle {  
        position: Point {  
            x: 10,  
            y: 20,  
        },  
    };  
  
    println!("Posición: ({}, {})", rectangulo.position.x,  
rectangulo.position.y);  
}
```

Salida:

```
Posición: (10, 20)
```

3.3.13. Funciones y parámetros

```

<function_decl> ::= fn <id> ( <param_list> ) -> <type>
                    { <stmt_list> }

<function_call> ::= <id> ( <arg_list> )

<param_list> ::= <param>
                | <param> , <param_list>

<param> ::= <id> : <type>

<return_stmt> ::= return <exp> ;

```

Una función es un bloque de código que realiza una tarea específica. Las funciones permiten organizar un programa en partes más pequeñas, reutilizar código y facilitar su mantenimiento.

En Rust, una función se declara con la palabra reservada `fn`, seguida del nombre de la función, una lista de parámetros y, opcionalmente, el tipo del valor que retorna.

La sintaxis general es la siguiente:

```

fn nombre(parametro: Tipo) -> TipoRetorno {
    // instrucciones
}

```

Si la función no devuelve un valor, el tipo de retorno puede omitirse.

Ejemplo:

```

fn saludar(nombre: String) {
    println!("Hola, {}", nombre);
}

fn sumar(a: i32, b: i32) -> i32 {
    return a + b;
}

fn main() {
    saludar(String::from("Ana"));

    let resultado = sumar(10, 5);
    println!("Resultado: {}", resultado);
}

```

Salida:

```
Hola, Ana
```

Resultado: 15

3.3.14. Funciones embebidas (incluye de los arreglos)

Las funciones embebidas son funciones que forman parte del lenguaje y pueden utilizarse directamente, sin necesidad de que el usuario las implemente. Estas funciones proporcionan operaciones comunes sobre los tipos de datos del lenguaje, como cadenas de texto, arreglos y operaciones de propósito general, facilitando el desarrollo de programas.

Función	Tipo	Descripción	Resultado de la llamada
<code>println!("Hola {}", nombre)</code>	Global	Imprime un mensaje o uno o varios valores en pantalla y agrega un salto de línea al finalizar. Los marcadores <code>{}</code> indican la posición donde se insertarán los valores, y las variables o expresiones que reemplazan cada marcador se escriben después de la cadena de texto, separadas por comas.	<code>Hola Pablo</code>
<code>typeof(valor)</code>	Global	Devuelve el tipo de dato de una expresión.	<code>"i32"</code>
<code>random(min, max)</code>	Global	Genera un número aleatorio dentro de un rango.	<code>7</code>
<code>len()</code>	String / Arreglo	Devuelve la longitud de una cadena o la cantidad de elementos de un arreglo.	<code>9</code>
<code>contains(valor)</code>	String / Arreglo	Indica si un elemento o una subcadena existe.	<code>true</code>
<code>replace(a, b)</code>	String	Reemplaza una parte del texto por otra.	<code>"Hi Mundo"</code>
<code>split(separador)</code>	String	Divide una cadena utilizando un separador.	<code>["Hola", "Mundo"]</code>
<code>to_uppercase()</code>	String	Convierte una cadena a mayúsculas.	<code>"HOLA MUNDO"</code>
<code>to_lowercase()</code>	String	Convierte una cadena a minúsculas.	<code>"hola mundo"</code>
<code>reverse()</code>	Arreglo	Invierte el orden de los elementos de un arreglo.	<code>[30, 20, 10]</code>

Ejemplo:

```
fn main() {
    let texto = String::from("Hola Mundo");
    let mut numeros = [10, 20, 30];

    // println!
    println!("Hola Mundo");
    println!(texto);
    println!("Texto: {}", texto);
    println!("Longitud: {}", texto.len());

    // Funciones de String
    println!(typeof(texto));
    println!(texto.len());
    println!(texto.contains("Mundo"));
    println!(texto.replace("Mundo", "Rust"));
    println!(texto.to_uppercase());
    println!(texto.to_lowercase());
    println!(texto.split(" "));

    // Funciones de arreglos
    numeros.reverse();
    println!(numeros);

    // Función aleatoria
    println!(random(1, 10));
}
```

Salida:

```
Hola Mundo
Hola Mundo
Texto: Hola Mundo
Longitud: 10
String
10
true
Hola Rust
HOLA MUNDO
hola mundo
["Hola", "Mundo"]
[30, 20, 10]
7
```

3.3.15. Función main

La función main es el punto de entrada de un programa. Cuando el programa se ejecuta, las instrucciones contenidas en esta función son las primeras en ejecutarse. Todo programa debe definir una única función main, ya que sin ella el sistema no sabría dónde iniciar la ejecución.

Dentro de la función main es posible:

- Declarar e inicializar variables.
- Crear arreglos, cadenas de texto y estructuras.
- Invocar funciones definidas por el usuario.
- Utilizar funciones embebidas del lenguaje.
- Controlar el flujo de ejecución mediante estructuras de control.

Una vez que todas las instrucciones de **main** han sido ejecutadas, el programa finaliza.

Ejemplo:

```
fn saludar(nombre: String) {
    println!("Hola " + nombre);
}

fn main() {
    let nombre = String::from("Ana");
    let edad = 20;

    println!(nombre);
    println!(edad);

    saludar(nombre);
}
```

Salida:

```
Ana
20
Hola Ana
```

3.4. Reportes

3.4.1. Reporte de errores

Manejo de errores:

El intérprete deberá detectar y reportar errores durante las diferentes fases del proceso de interpretación. Para ello se consideran tres categorías de errores: léxicos, sintácticos y semánticos.

Cada error deberá incluir información suficiente para que el usuario pueda identificar fácilmente su origen, indicando como mínimo:

- Tipo de error.
- Descripción del problema.
- Línea donde ocurrió.
- Columna (posición dentro de la línea).
- Fragmento del código donde se detectó.

El formato propuesto para los mensajes de error es el siguiente:

```
[Tipo de Error] Línea <línea>, Columna <columna>  
Descripción del error.
```

Ejemplo:

```
[Error Léxico] Línea 3, Columna 15  
Carácter no reconocido: '@'
```

Tipos de errores que se deben de identificar:

1. Error Carácter no reconocido

Ocurre cuando aparece un símbolo que no forma parte del lenguaje.

Ejemplo

```
let total = @100;
```

Salida

```
[Error Léxico] Línea 1, Columna 13  
Carácter no reconocido '@'.
```

2. Falta de punto y coma

Ejemplo

```
let x = 10  
println!(x);
```

Salida

```
[Error Sintáctico] Línea 1, Columna 11  
Se esperaba ';'.
```


3. Llave o paréntesis sin cerrar

Ejemplo

```
fn main() {  
    println!("Hola");
```

Salida

```
[Error Sintáctico] Línea 2, Columna 20  
Se esperaba '}'.
```

4. Variable no declarada

Ejemplo

```
println!(total);
```

Salida

```
[Error Semántico] Línea 1, Columna 9  
La variable 'total' no ha sido declarada.
```

5. Tipos incompatibles

Ejemplo

```
let edad: i32 = "veinte";  
let x: i32 = 10;  
let y = x + "5";
```

Salida

```
[Error Semántico] Línea 1, Columna 17  
No es posible asignar un valor de tipo String a una variable de tipo  
i32.
```

```
[Error Semántico] Línea 3, Columna 12  
Tipos incompatibles. No es posible utilizar un valor de tipo String  
donde se esperaba un valor de tipo i32.
```

6. Modificación de una variable inmutable

Ejemplo

```
let contador = 10;  
  
contador = 20;
```

Salida

```
[Error Semántico] Línea 3, Columna 1  
No es posible modificar la variable 'contador' porque fue declarada como  
inmutable.
```

7. Acceso fuera de los límites de un arreglo

Ejemplo

```
let numeros = [10, 20, 30];  
  
println!(numeros[5]);
```

Salida

```
[Error Semántico] Línea 3, Columna 10  
Índice fuera de los límites del arreglo.
```

8. Función no declarada

Ejemplo

```
saludar();
```

Salida

```
[Error Semántico] Línea 1, Columna 1  
La función 'saludar' no ha sido declarada.
```

9. Número incorrecto de parámetros

Ejemplo

```
fn suma(a: i32, b: i32) {  
}
```

```
fn main() {
    suma(10);
}
```

Salida

```
[Error Semántico] Línea 5, Columna 5
La función 'suma' esperaba 2 argumentos, pero recibió 1.
```

Resiliencia durante la ejecución:

Los mensajes de error mostrados en los ejemplos anteriores representan únicamente el reporte del error detectado y no implican que el programa deba detener su ejecución. El intérprete deberá intentar recuperarse y continuar con el análisis y la ejecución del resto del código siempre que el error no sea crítico. Esto permitirá detectar y reportar múltiples errores en una sola ejecución. Solo los errores que impidan continuar el análisis de forma segura podrán provocar la finalización del programa. Esto es fundamental ya que los errores deberán guardarse y **generar una tabla de análisis de errores**.

Ejemplo:

No	Tipo	Descripción	Línea	Columna
1	Léxico	Carácter no reconocido @.	3	15
2	Sintáctico	Se esperaba ; al final de la instrucción.	7	18
3	Semántico	La variable contador no ha sido declarada.	12	9
4	Semántico	No es posible asignar un valor de tipo String a una variable de tipo i32.	18	21

3.4.2. Reporte tabla de símbolos

La tabla de símbolos es una estructura utilizada por el intérprete para almacenar la información de los identificadores declarados durante el análisis del programa. Este reporte permitirá visualizar el estado de todos los símbolos registrados, facilitando la verificación del análisis semántico y la depuración del código.

Al finalizar la ejecución, el intérprete deberá generar automáticamente un reporte en formato HTML con la información de cada símbolo registrado.

Ejemplo:

No.	Identificador	Categoría	Tipo	Ámbito	Línea	Valor
-----	---------------	-----------	------	--------	-------	-------

1	contador	Variable	i32	Global	1	10
2	nombre	Variable	String	Global	2	"Ana"
3	sumar	Función	i32	Global	5	—
4	persona	Struct	Person	Global	12	—
5	edad	Variable	i32	sumar	6	20

3.4.3. Reporte de AST

El Árbol de Sintaxis Abstracta (Abstract Syntax Tree, AST) es una representación jerárquica del programa obtenida durante el análisis sintáctico. A diferencia del árbol de derivación, el AST conserva únicamente la información relevante para representar la estructura lógica del programa, eliminando los elementos propios de la sintaxis que no aportan significado semántico.

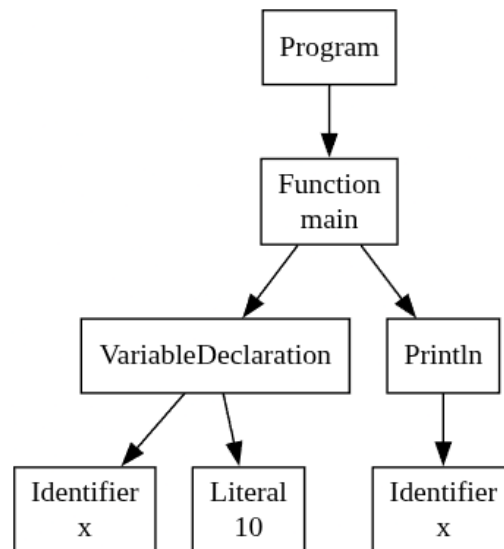
Como parte del proyecto, el estudiante deberá implementar su propia estructura de nodos para representar el AST. Aunque PLY proporciona las herramientas necesarias para construir el analizador sintáctico, no genera automáticamente un Árbol de Sintaxis Abstracta, por lo que será responsabilidad del estudiante diseñar las clases o estructuras de datos que representen cada uno de los elementos del lenguaje.

Al finalizar la ejecución, el intérprete deberá generar automáticamente un reporte del AST utilizando Graphviz, construyendo el archivo fuente en formato DOT y generando su correspondiente representación gráfica (PNG, SVG o PDF). Este reporte permitirá verificar visualmente la estructura interna del programa y facilitará la depuración del intérprete.

Ejemplo:

```
fn main() {
    let x = 10;
    println!(x);
}
```

Salida:



3.5. Alcance del proyecto

Para que los estudiantes sepan hasta dónde deben llegar para una correcta calificación se establecen los siguientes alcances.

Alcance obligatorio (Funcionalidades esenciales)

- Editor de código: La aplicación debe permitir crear, abrir, editar y guardar archivos.
- Análisis léxico y sintáctico para el lenguaje de entrada.
- Análisis semántico: validación de tipos, declaración y alcance (scope) de identificadores, manejo de entornos.
- Ejecución: el intérprete localiza la función principal y la ejecuta.

Realizar todas las **operaciones aritméticas, lógicas, declaración de variables con todos los tipos, función de imprimir en consola**, ya que será lo mínimo en todos los archivos de entrada de calificación que se va a utilizar.

3.6. Entregables

Tipo	Descripción
Prototipo	Desarrollo de una versión funcional inicial del proyecto.
Documentación Técnica	Documento en Markdown que describa: • Gramática formal de OxygenScript • Métodos utilizados para generar código a bajo nivel. • Decisiones de diseño y desafíos enfrentados

Código Fuente	Repositorio Git organizado (GitHub) con todo el código. El nombre del repositorio debe seguir el siguiente formato: OLC2_[Seccion]_P1_2S2026
Manual de Usuario	- Manual paso a paso para instalar y usar la herramienta - Instrucciones para crear, editar y ejecutar código, así como para interpretar los reportes (AST, tabla de símbolos y errores). - Capturas de pantalla de la interfaz y ejemplos de sesión de uso.

Se debe agregar al auxiliar designado como colaborador en el proyecto.

Usuarios de Github:

Sección: Auxiliar	Github
A: Diego Cali	DiegoCali
B: Diego Dubón	diegodubon31
N: Omar Vides	AllVides

4. Material de apoyo

La documentación de la herramienta para generar el análisis léxico y sintáctico se encuentra en este link <https://www.dabeaz.com/ply/ply.html>. También puedes consultar ejemplos y ejercicios en el [repositorio del laboratorio](#).

5. Metodología

1. **Investigación preliminar:** Revisar conceptos de teoría de compiladores como el análisis léxico, sintáctico y semántico. Estudiar ejemplos básicos de gramática. Configurar el entorno para ejecutar el programa y repositorio Git.
2. **Diseño del sistema:** Definir la gramática formal, cubriendo variables, estructuras de control, arreglos, funciones etc. Documentar la estructura de carpetas y el patrón arquitectónico a utilizar o flujo del desarrollo.
3. **Desarrollo:**
 - a. **Bloque 1:** Implementar y probar el analizador léxico.
 - b. **Bloque 2:** Generar el parser y construir el AST.
 - c. **Bloque 3:** Implementar el análisis semántico (tabla de símbolos, validaciones de tipos). Crear casos de prueba para semántica y manejo de errores.
 - d. **Bloque 4:** Desarrollar la GUI. Integrar el intérprete con la interfaz.

4. **Pruebas y ajustes:** Se realizarán pruebas para verificar que la aplicación funciona correctamente en diferentes condiciones.

6. Recursos y herramientas a utilizar

Los estudiantes deberán emplear las siguientes tecnologías y herramientas en el desarrollo:

- Lenguaje de entrada: OxygenScript (Rust)
- Lenguaje de implementación: Python
- Generación de analizadores léxico y sintáctico: PLY.
- Interfaz gráfica de usuario: Django, Graphviz.

7. Desarrollo de Habilidades Blandas

En los proyectos individuales, cada estudiante asume plenamente la planificación, ejecución y entrega, promoviendo no solo habilidades técnicas sino también Profesionales y de aprendizaje continuo.

6.1.1 Gestión del tiempo

Planificar tareas y establecer hitos propios para asegurar el avance constante y la entrega puntual de cada fase del intérprete.

6.1.2 Autonomía en la investigación

Buscar y seleccionar recursos, ejemplos de gramáticas y buenas prácticas para resolver dudas y guiar el diseño sin depender de supervisión constante.

6.1.3 Resolución creativa de problemas

Enfrentar errores en la gramática, el parser o la lógica de ejecución con iniciativas propias: diseñar casos de prueba, depurar paso a paso y ajustar la implementación.

6.1.4 Calidad de la documentación y pruebas

Mantener un README claro, escribir casos de prueba significativos y documentar cada módulo (lexer, parser, AST, semántico, GUI) para facilitar el mantenimiento y la evaluación.

8. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Elaboración	24/07/2026	28/08/2026
Calificación	31/08/2026	05/09/2026

9. Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado en lenguaje Python , usando PLY , Django/React/Angular/etc. para la GUI y sistema operativo Linux .	Si
Gestión y entregas del proyecto	- Todos los commits realizados disponibles en el repositorio GitHub.	Si
Documentación obligatoria	- Manual de usuario. - Diagrama de clases y de flujo de procesamiento (AST, tabla de símbolos).	Si
Pruebas y funcionalidad mínima	- El intérprete ejecuta correctamente los elementos obligatorios (definido en la sección de alcance obligatorio).	Si
Modificación de código	- Se indicará al estudiante que modifique parte del código; en caso de no lograrlo, no podrá ser calificado.	Sí